

Les codes à usage unique, Présentation et mise en œuvre



SOMMAIRE

- Présentation
- Rappel des fondamentaux de sécurité
- TOTP : application et usage
- De la théorie...
- ...à la pratique
- Échecs, vulnérabilités et tromperies
- Et quoi d'autre ?

PRÉSENTATION

OSEF ?

Mail : tenshiphe [at] mailfence [dot] com

DISCLAIMER





RAPPEL DES FONDAMENTAUX DE SÉCURITÉ

SÉQUENCE DE BASE

Les trois phases d'une validation d'identité :

- 1) Identification
- 2) Authentification
- 3) Accréditation

TYPES DE PREUVE

Quatre types de preuve :

- Ce que l'on est
- Ce que l'on sait
- Ce que l'on possède
- Ce que l'on sait faire

LES METHODES

Méthodes couramment employés :

- Mot de passe, passe-phrase
- Certificat
- Biométrie
- Token à usage unique

LES TYPES DE SECRETS

	Secret gardé	Secret partagé	Secret réparti
Nombre de détenteur	1	2	n
Usage	simple	moyen	complexe
Sécurisation	bonne	modérée	forte
Résilience	faible	moyenne	forte
Exemple	Outil local sans accès Internet	Accès à un site Internet	PKI (clés Shamir)

TOTP, APPLICATION ET USAGE

ORIGINE

OATH : Initiative for Open Authentication (Initiative pour l'authentification ouverte)

HOTP : un algorithme de mot de passe à usage unique basé sur HMAC (RFC 4226)

TOTP : Algorithme de mot de passe à usage unique basé sur le temps (RFC 6238)

NE PAS CONFONDRE

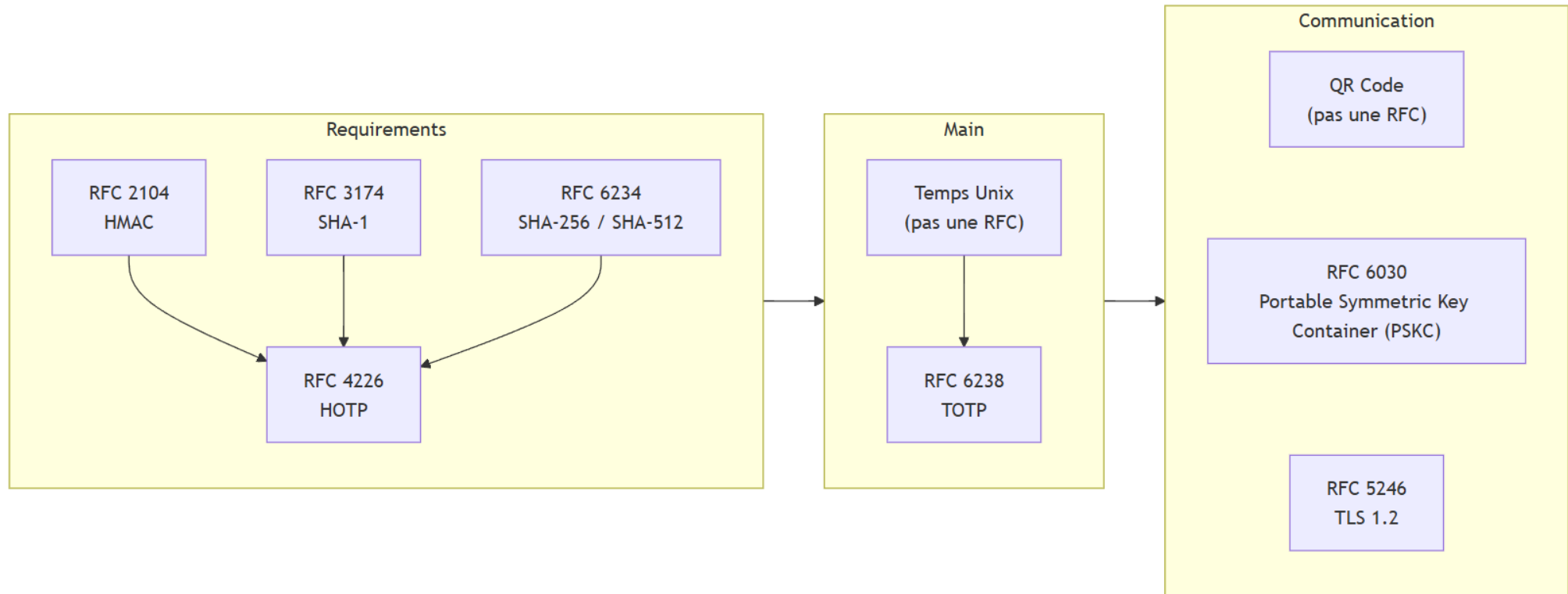
- OAuth est un protocole de gestion des autorisation de l'IETF



- TOTP n'est pas une émission télé



DIAGRAMME DES RFC



COMPARAISON

Caractéristique	TOTP	HOTP
Méthode de génération	Mot de passe à usage unique basé sur le temps.	Mot de passe à usage unique basé sur un compteur.
Durée de validité	Utilise l'heure actuelle comme entrée.	Utilise un compteur qui s'incrémente à chaque utilisation.
Synchronisation	Généralement valide pour une courte période (ex. 30 secondes).	Valide jusqu'à ce qu'il soit utilisé ou qu'un nouveau mot de passe soit généré.
Utilisation	Applications mobiles et systèmes en ligne.	Systèmes où les demandes de mot de passe sont moins fréquentes.
Sécurité	Plus sécurisé contre les attaques par rejeu (durée de validité courte).	Moins sécurisé contre les attaques par rejeu si le mot de passe n'est pas utilisé rapidement..
Cas d'usage	Google Authenticator, Authy, Bastion, SSH / PAM	Systèmes de sécurité basés sur des jetons matériels.

SCHÉMA DE FONCTIONNEMENT

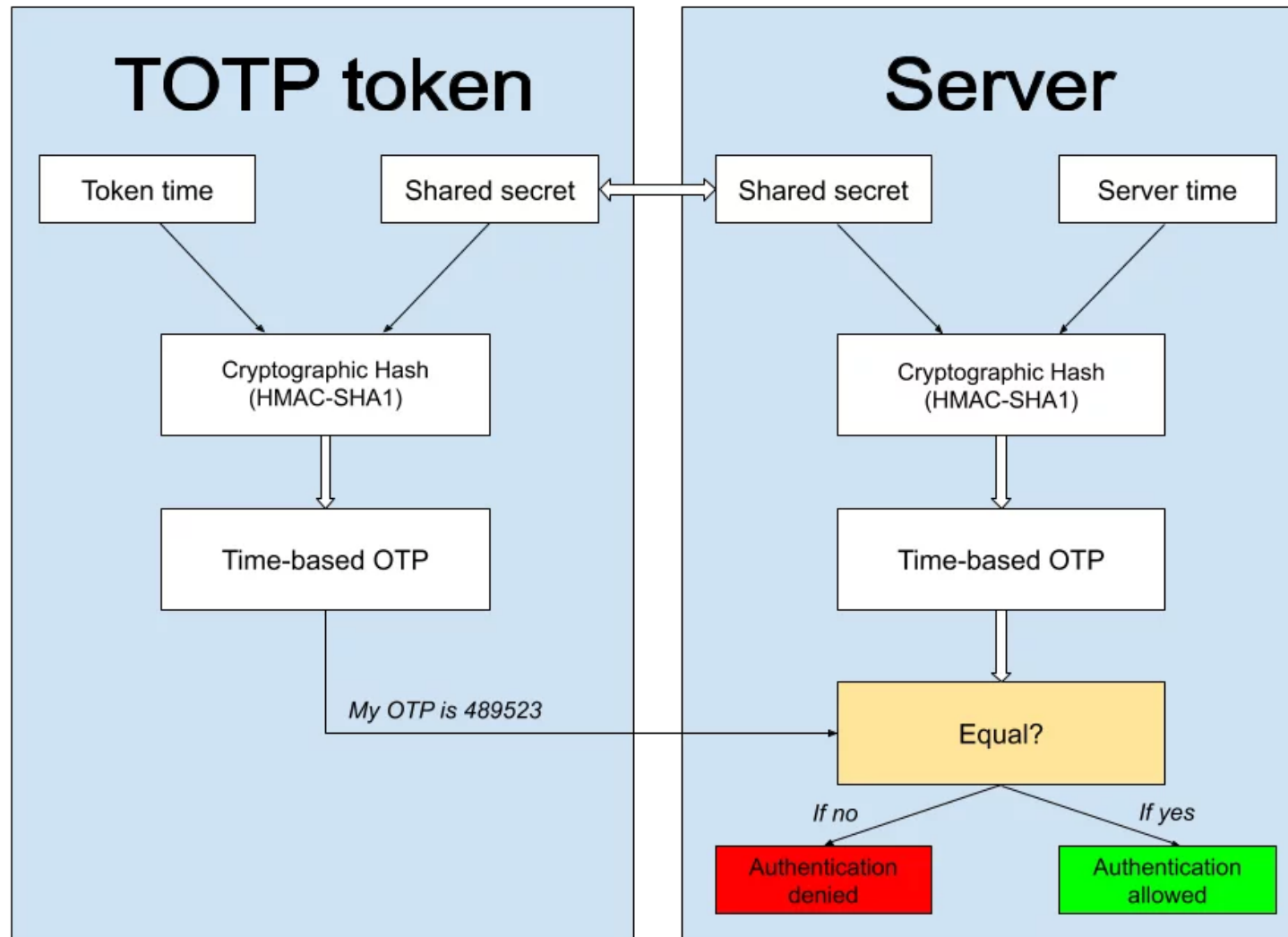


Schéma #1

SCHÉMA D'UTILISATION

Setup Two-Factor Authentication

1. Install

To use two-factor authentication, you will need to download the SecureAuth mobile app to your smart phone



You can also use Google Authenticator:

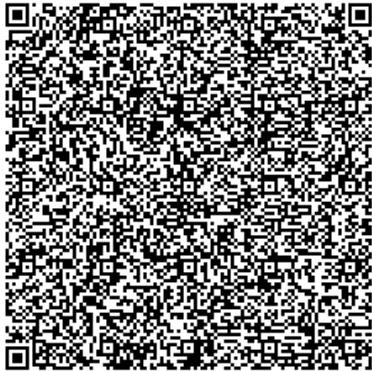
[Apple App Store](#) | [Google Play](#)

or any app that supports

[RFC 6238](#)

2. Scan

Open your two-factor authentication app and scan the code with the camera on your phone.



Q7KF2Y

'U74XS

You may also enter the code manually in Google Authenticator or other two-factor apps

3. Confirm

Enter the verification code generated by your two-factor authentication app.

☐ Enter custom app name

Schéma #2

APPLICATIONS CLIENTES

- Google Authenticator
- 2FA Authenticator
- Ente Auth
- Microsoft Authenticator
- Authy
- Aegis Authenticator
- Yubico Authenticator
- ...
- Gestionnaire de mot de passe (!)

APPLICATIONS SERVEURS

On Cloud :

- EntraID
- Okta
- CyberArk
- Google Cloud Identity Platform
- ...

On Premise :

- Payante :
Duo security,
Keycloak,
...
- Gratuite :
OATHTools,
Google Authenticator PAM

SDK :

- Protector OATH SDK for Android
- Apple's Security Framework
- Dépôts Git (!)

DE LA THÉORIE...

FORMULE

$$TOTP = \text{mod} \left(\text{Truncate} \left(\text{HMAC}_{\text{SHA1}}(K, T) \right), 10^d \right)$$

$$T = \left\lfloor \frac{\text{UnixTime}(\text{now}) - \text{UnixTime}(T_0)}{T_x} \right\rfloor$$

$$H = \text{HMAC}_{\text{SHA1}}(K, T)$$

- K est la clé secrète partagée
- T est le compteur de temps
- d est la longueur du code TOTP (généralement 6)

FORMULE

$$T = \left\lfloor \frac{UnixTime(now) - UnixTime(T_0)}{T_x} \right\rfloor$$

- UnixTime(now) est le temps actuel, en secondes, depuis le 1er janvier 1970
- T₀ est le compteur de temps (généralement 0)
- T_x est la durée de l'intervalle (généralement 30)

A la date du 9 septembre 2001 à 01:46:40 UTC, avec des intervalles de 30 secondes, l'horodatage est est 1 000 000 000. Nous avons donc T = 1000000000/30, soit 33 333 3333,33 qui nous donne 33 333 333.

FORMULE

$$T = \left\lfloor \frac{UnixTime(now) - UnixTime(T_0)}{T_x} \right\rfloor$$

- Conversion de 33 333 3333 en binaire big endian

3 333 333 $\div 2 = 16\,666\,666$ reste 1

16 666 666 $\div 2 = 8\,333\,333$ reste 0

...

3 $\div 2 = 1$ reste 1

1 $\div 2 = 0$ reste 1



Lecture des restes de bas en haut : 1111111001010000001010101

- Valeur en hexadécimal : 1FCA055
- Big endian (les octets les plus significatifs sont à gauche) : 01FCA055

FORMULE

$$T = \left\lfloor \frac{UnixTime(now) - UnixTime(T_0)}{T_x} \right\rfloor$$

En bref :

Format	Valeur
T (décimal)	33 333 333
T (hexadécimal brut)	1FCA055
T (hex, 8 octets)	0000000001FCA055
T (en octets)	[00, 00, 00, 00, 01, FC, A0, 55]

FORMULE

$$H = HMAC_{SHA1}(K, T)$$

- K : pas de contrainte dans la RFC. L'emploi base32 est devenu un standard :
 - 16 à 128 caractères
 - lettres A-Z sans I, L, O
 - chiffres 2-7

Format	Valeur
K (base32)	ACFSECUFYSYK2K25
K (hexadécimal)	00 8B 22 0A 85 C4 B0 AD 2B 5D
K (5 bits)	00000 00010 00101 10010 00100 00010 10100 00101 11000 10010 11000 01010 11010 01010 11010 11101
K (8 bits)	00000000 10001011 00100010 00001010 10000101 11000100 10110000 10101101 00101011 01011101
K (10 octets)	[00, 8B, 22, 0A, 85, C4, B0, AD, 2B, 5D]
K (64 octets)	[00, 8B, 22, 0A, 85, C4, B0, AD, 2B, 5D, 00, 00, ..., 00]

FORMULE

RAPPEL

AND : Renvoie vrai uniquement si les deux opérandes sont vraies.

XOR : Renvoie vrai si exactement une des opérandes est vraie.

A	0	0	1	1
B	0	1	0	1
AND	0	0	0	1
XOR	0	1	1	0

FORMULE

$$H = \text{HMAC}_{\text{SHA1}}(K, T)$$

$$\text{HMAC}(K, T) = \text{SHA1}((K \oplus \text{opad}) \parallel \text{SHA1}((K \oplus \text{ipad}) \parallel T))$$

- opad (outer padding) : l'octet 5C répété 64 fois
- ipad (inner padding) : l'octet 36 répété 64 fois
- $\parallel$ signifie concaténation
- $\oplus$ est l'opérateur XOR

FORMULE

$$H = HMAC_{SHA1}(K, T)$$

$$HMAC(K, T) = SHA1((K \oplus \text{opad}) \parallel SHA1((K \oplus \text{ipad}) \parallel T))$$

key	00	8B	22	0A	85	C4	B0	AD	2B	5D
opad	5C	5C	5C	5C	5C	5C	5C	5C	5C	5C
result	5C	D7	7E	56	D9	98	EC	F0	77	01

key	00	8B	22	0A	85	C4	B0	AD	2B	5D
ipad	36	36	36	36	36	36	36	36	36	36
result	36	BD	14	3C	B3	F2	86	9B	1D	6B

FORMULE

$$H = HMAC_{SHA1}(K, T)$$

$$HMAC(K, T) = SHA1((K \oplus \text{opad}) \parallel SHA1((K \oplus \text{ipad}) \parallel T))$$

$$\begin{aligned} &SHA1(K \oplus \text{ipad} \parallel T) = \\ &60DAA0CC3017B3328ED3D5B980800453274B2A78 \end{aligned}$$

$$\begin{aligned} &SHA1(K \oplus \text{opad} \parallel \text{inner_hash}) = \\ &49EE4286C4B9E6D0CA397CDCB599898743ED5606 \end{aligned}$$

On prend la clé K,
on la mélange avec deux constantes (ipad et opad),
on fait deux SHA1 successifs,
et on obtient le HMAC.

FORMULE

$$H = \text{HMAC}_{\text{SHA1}}(K, T)$$

- Calcul de l'offset :

On prend le dernier octet du HMAC (ici 0x06) et on fait un ET logique :

Hex	Binaire								Dec
06	0	0	0	0	0	1	1	0	06
0F	0	0	0	0	1	1	1	1	14
06	0	0	0	0	0	1	1	0	06

FORMULE

$$H = \text{HMAC}_{\text{SHA1}}(K, T)$$

- Calcul de l'offset :

On ignore les 4 premiers octets puis, on extrait les 4 octets depuis l'offset en 6e position, soit les octets 10, 11, 12, 13 du HMAC.

Position	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
HMAC-SHA1	49	ee	42	86	c4	b9	e6	d0	ca	39	7c	dc	b5	99	89	87	43	ed	56	06

Cette technique, appelée troncature dynamique, vise à rendre la position d'extraction moins prévisible et donc à augmenter la sécurité du code généré.

FORMULE

$$TOTP = \text{mod} \left(\text{Truncate} \left(\text{HMAC}_{SHA1}(K, T) \right), 10^d \right)$$

On multiplie la valeur de chaque octet par 2 avec sa position en puissance :

$$\text{entier} = 7C \times 2^{24} + DC \times 2^{16} + B5 \times 2^8 + 99 \times 2^0$$

$$= 124 \times 16\,777\,216 + 220 \times 65\,536 + 181 \times 256 + 153 \times 1$$

$$= 2\,091\,692\,672 + 14\,421\,760 + 46\,336 + 153$$

$$= 2\,095\,959\,289$$

FORMULE

$$TOTP = \text{mod} \left(\text{Truncate} \left(\text{HMAC}_{\text{SHA1}}(K, T) \right), 10^d \right)$$

$$\text{TOTP} = \text{mod} (2095959289 , 10^6)$$

$$\text{TOTP} = 2\,095\,959\,289 \div 1\,000\,000$$

$$\text{TOTP} = 2095.959289$$

$$\text{TOTP} = 959\,289$$

```
tenshiphe@SYRIAL ~$ oathtool --totp --base32 ACF5ECUFYSYK2K25 --now '2001-09-09 01:46:40 UTC'
959289
```


STATISTIQUES

Sur un référentiel d'une occurrence de chaque valeur possible par longueur

DURÉE	QUANTITÉ	LONGUEUR	FRÉQUENCE D'APPARITION	NOMBRE D'OCCURRENCE
1 jour	2 880			
1 semaine	20 160	6	600 000	100 000
1 mois	86 400	7	7 000 000	1 000 000
1 an	1 051 200	8	80 000 000	10 000 000

LONGUEUR	NOMBRE DE COMBINAISON	UNIQUE		DOUBLE		TRIPLE	
		NB	%	NB	%	NB	%
6	1 000 000	151 200	15,12	848 800	84,88	100 800	10,08
7	10 000 000	604 800	6,04	9 395 200	93,95	1 058 400	10,58
8	100 000 000	1 814 400	1,81	98 185 600	98,18	8 467 200	8,46

ANALYSE FREQUENTIELLE

Fréquence d'apparition et tableau des digrammes de notre secret partagé sur la journée du 19 juillet 2025

0	1721
1	1771
2	1705
3	1746
4	1675
5	1873
6	1724
7	1685
8	1718
9	1668

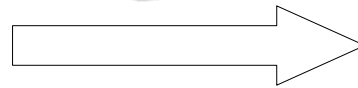
	0	1	2	3	4	5	6	7	8	9
0	89	91	89	64	75	100	100	69	91	91
1	91	91	95	66	86	99	76	87	73	76
2	89	88	75	91	81	97	94	92	73	88
3	64	66	91	97	93	104	97	76	80	76
4	81	79	95	79	82	93	95	67	101	94
5	93	92	77	88	84	100	86	87	106	105
6	95	91	94	90	84	88	88	75	95	90
7	84	100	88	95	83	102	85	75	97	87
8	81	84	73	80	73	92	95	97	87	79
9	76	93	88	68	74	92	90	87	79	74

...À LA PRATIQUE

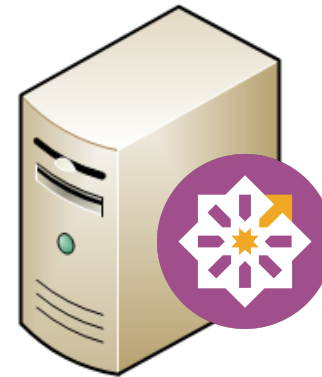
SCENARIO



Debian
client

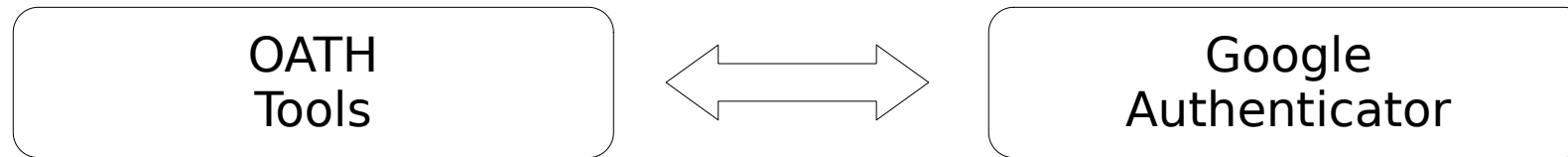


SSH
Password + TOTP



CentOS Stream
server

REPOSITORY & PACKAGE



EPEL

```
yum update  
yum -y install epel-release  
yum repolist  
yum install oathtool pam_oath
```

REPOSITORY & PACKAGE

Contenu du package :

- **liboath**
- **oathtool**
- **pam_oath**
- **libpskc**
- **pskctool**

REPOSITORY & PACKAGE

Fichiers de configuration :

Dans « /etc/ »,

- security/users.oath**
- pam.d/sshd**
- ssh/sshd_config.d/50-redhat.conf**

FICHER USERS.OATH

/etc/security/users.oath

```
# Syntax : <algorithm>/<counterInfo>/<digit> <user> <PIN> <token>  
HOTP/T30 bob - ACFSECUFYSYK2K25
```

Sécurisation du fichier :

```
chmod 600 /etc/security/users.oath  
chown root:root /etc/security/users.oath
```

<https://code.google.com/archive/p/mod-authn-otp/wikis/UsersFile.wiki>

FICHER PAM

/etc/pam.d/sshd

```
auth required pam_oath.so usersfile=/etc/security/users.oath window=5 digits=6
```

Pour limiter l'usage du TOTP à un groupe d'utilisateur :

```
auth [default=1 success=ignore] pam_succeed_if.so quiet user ingroup oath  
auth requisite pam_oath.so usersfile=/etc/users.oath window=20 digits=8
```

FICHER SSH

/etc/ssh/sshd_config.d/50-redhat.conf

< OpenSSH 8.7p1

```
UsePAM yes  
ChallengeResponseAuthentication yes  
PasswordAuthentication yes  
AuthenticationMethods keyboard-interactive;pam
```

≥ OpenSSH 8.7p1

```
KbdInteractiveAuthentication yes
```

PREUVE DE CONNEXION

```
$ ssh sparadrap@192.168.0.169
(sparadrap@192.168.0.169) One-time password (OATH) for `sparadrap':
(sparadrap@192.168.0.169) Password:
Last login: Sun Mar  8 22:58:34 2026 from 127.0.0.1
sparadrap@localhost:~$
```



ÉCHECS, VULNÉRABILITÉS ET TROMPERIES

MAUVAISE UTILISATION

Setup Two-Factor Authentication

1. Install

To use two-factor authentication, you will need to download the SecureAuth mobile app to your smart phone



You can also use Google Authenticator:

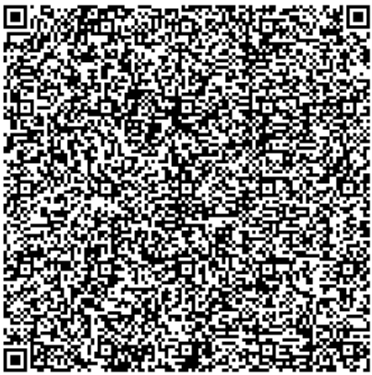
[Apple App Store](#) | [Google Play](#)

or any app that supports

[RFC 6238](#)

2. Scan

Open your two-factor authentication app and scan the code with the camera on your phone.



Q7KF2Y

'U74XS

You may also enter the code manually in Google Authenticator or other two-factor apps

3. Confirm

Enter the verification code generated by your two-factor authentication app.

☐ Enter custom app name

Schéma #2

MAUVAISE IMPLÉMENTATION

Les erreurs récurrentes, coté serveur :

- **pas de sauvegarde**
- **pas de bris-de-glace**
- **pas de test des procédures**
- **...**

Les erreurs récurrentes, coté client :

- **attention à la synchronisation du temps !**
- **vérification du chiffrement de la connexion**
- **absence de chiffrement du stockage**
- **sauvegarde du code de recouvrement**

VULNERABILITÉ

Mauvaise implémentation :

- **CVE-2024-43491 : AuthQuake (Microsoft)**

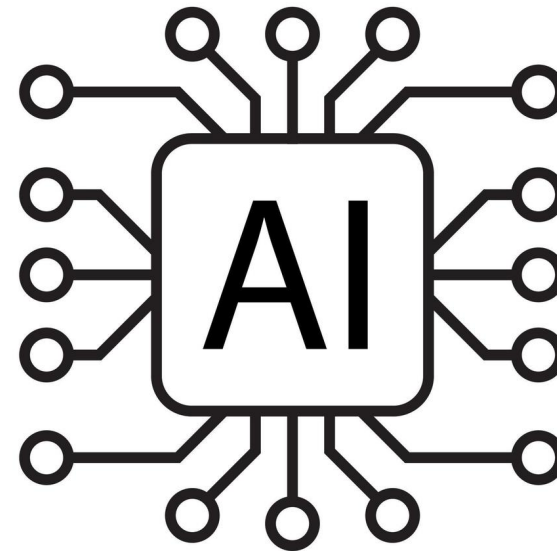
Exploit hardware :

- **CVE-2025-48561 : Pixnapping (Google/Android)**

Phishing :

- **CVE-2017-18948 : SS7 (Telefonica, Allemagne)**

LA TROMPERIE



Le CLUSIF parle des attaques via SMS depuis au moins 2014...

ET QUOI D'AUTRE ?

DE NUMÉRIQUE À PHYSIQUE



DE NUMÉRIQUE À PHYSIQUE

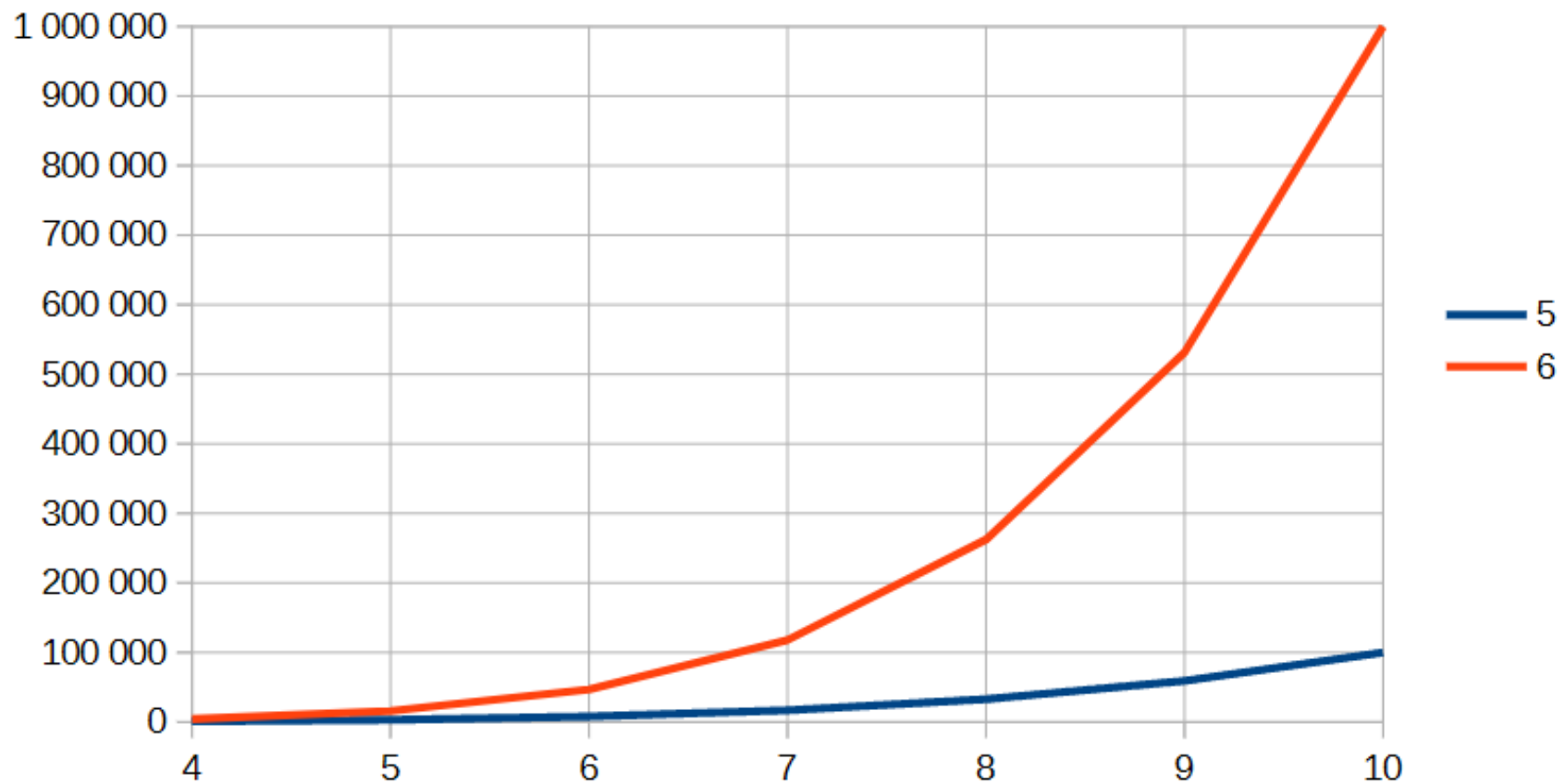
p^g = nombre de combinaisons

NOMBRE DE GOUPILLE	NOMBRE DE POSITION	NOMBRE DE COMBINAISON
5	4	1 024
	5	3 125
	6	7 776
	7	16 807
	8	32 768
	9	59 049
	10	100 000
6	4	4 096
	5	15 625
	6	46 656
	7	117 649
	8	262 144
	9	531 441
	10	1 000 000

DE NUMÉRIQUE À PHYSIQUE

p^g = nombre de combinaisons

Evolution du nombre de combinaison par taille de clé



LA SÉCURITÉ FRANÇAISE

Vol de donnée en France en 2025 :

- **1^{er} place européenne, et 2^e mondiale !**

Cambriolage en France en 2024 :

- **2^e place européenne (et 12^e mondiale en 2022)**

Cambriolage en France en 2025 (estimation) :

- ***1^{er} place européenne, et 4^e mondiale***

CONCLUSION

Apprendre tous les jours
Maîtriser ses outils



**Privilégier la souveraineté numérique
individuelle et hors ligne !**

REMERCIEMENTS

Merci de votre attention !

Avez-vous des questions ?



ANNEXES

SOURCES 1/2

Wikipédia : <https://fr.wikipedia.org>

Open Authentication : <https://openauthentication.org>

Calcul et conversion : <https://www.rapidtables.com>

Dcode : <https://www.dcode.fr/code-base-32>

GeekForGeek : <https://www.geeksforgeeks.org/dsa/bit-manipulation-swap-endianness-of-a-number/>

Korben : <https://korben.info/gpu-zip-pixnapping-android-2fa-faille-non-patchee.html>

IT-Connect : <https://www.it-connect.fr/authquake-une-faille-critique-decouverte-dans-le-mfa-de-microsoft/>

IProov : <https://www.iproov.com/fr/blog/one-time-passcode-otp-authentication-risks>

Linkedin : <https://www.linkedin.com/feed/update/urn:li:activity:7411305792072597504/>

CLUSIF : <https://www.globalsecuritymag.fr/CLUSIF-panorama-de-la,20150119,50072.html>

Schéma :

- #1 : <https://goldcod3.github.io/project/proyect-ft-otp>

- #2 : <https://docs.secureauth.com/2307/en/multi-factor-app-enrollment-qr-code-configuration.html>

SOURCES 2/2

RFC :

- **2104 : HMAC: Keyed-Hashing for Message Authentication**
- **4226 : HOTP: An HMAC-Based One-Time Password Algorithm**
- **4648 : The Base16, Base32, and Base64 Data Encodings**
- **6238 : TOTP: Time-Based One-Time Password Algorithm**
- **6030 : Portable Symmetric Key Container (PSKC)**
- **8332 : Use of RSA Keys with SHA-256 and SHA-512 in the Secure Shell (SSH) Protocol**